

Aurory

Audit



Presented by:

OtterSec contact@osec.io

Ajay Kunapareddy d1r3wolf@osec.io

Robert Chen r@osec.io



Contents

01 Executive Summary **2**

Overview . . . 2

Key Findings . . . 2

02 Scope **3**

03 Findings **4**

04 Vulnerabilities **5**

OS-AUR-ADV-00 [med] | Incorrect Account And Bump . . . 6

OS-AUR-ADV-01 [med] | Signature Omits Remaining Amount . . . 7

OS-AUR-ADV-02 [med] | Centralization Risk . . . 8

OS-AUR-ADV-03 [low] | Inaccurate Index Searching . . . 9

OS-AUR-ADV-04 [low] | Prevent Reentrancy . . . 10

05 General Findings **11**

OS-AUR-SUG-00 | Add Seed Check . . . 12

OS-AUR-SUG-01 | Avoid Zero Transfer . . . 13

OS-AUR-SUG-02 | Account Size Calculation . . . 14

OS-AUR-SUG-03 | Use Prefix String Seed . . . 15

OS-AUR-SUG-04 | Replace Hard-Coded Values With Constants . . . 16

OS-AUR-SUG-05 | Eliminate Unutilized Code . . . 17

Appendices

A Vulnerability Rating Scale **18**

B Procedure **19**

© 2023 Otter Audits LLC. All Rights Reserved.

01 | Executive Summary

Overview

Aurory engaged OtterSec to perform an assessment of the `casier-evm` and `casier-solana` programs. This assessment was conducted between July 3rd and July 6th, 2023. For more information on our auditing methodology, see Appendix B.

Key Findings

Over the course of this audit engagement, we produced 11 findings in total.

In particular, we discovered a locker bypass vulnerability due to lacking critical validations and a token account manipulation vulnerability, and a lack of user balance checks (OS-AUR-ADV-02). We also discovered a denial of service from utilizing incorrect parameters (OS-AUR-ADV-00) and omitted data in the admin signature (OS-AUR-ADV-01).

Additionally, we made recommendations around including an address check using a program and seeds (OS-AUR-SUG-00), saving gas by preventing zero transfers (OS-AUR-SUG-01), and improving code efficiency and clarity (OS-AUR-SUG-02).

© 2023 Otter Audits LLC. All Rights Reserved.

02 | Scope

The source code was delivered to us in a git repository at github.com/Aurory-Game/ocil and https://gitlab.com/aurory/blockchain/syncspace-evm. This audit was performed against commit 45e5e5b and 689cf63.

A brief description of the programs is as follows.

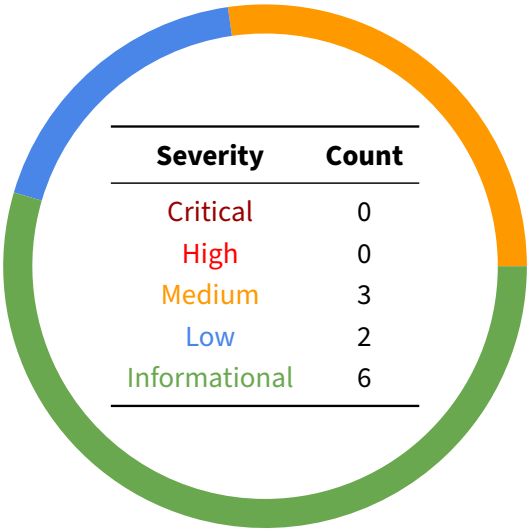
Name	Description
casier-evm	Creates the AURY token contract utilizing ERC20 and creates the contract for asset creation used in Aurory utilizing ERC1155. In particular, `SynsSpace` manages the admin escrow for both ERC20 and ERC1155, requiring an admin signature for access with `nonce` utilized to prevent double-spending.
casier-solana	Introduces a locking mechanism through `Locker` to manage users' assets. Depositing tokens into a `Locker` represents locking them, while withdrawals involve unlocking the tokens with the `before_amount` parameter to prevent double-spending.

© 2023 Otter Audits LLC. All Rights Reserved.

03 | Findings

Overall, we reported 11 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will help mitigate future vulnerabilities.



Severity	Count
Critical	0
High	0
Medium	3
Low	2
Informational	6

© 2023 Otter Audits LLC. All Rights Reserved.

04 | Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-AUR-ADV-00	Medium	Resolved	When `withdraw_from_burner_ta` is set to true, `Withdraw_v2` may result in a runtime error or a denial of service due to incorrect accounts and bumps.
OS-AUR-ADV-01	Medium	Resolved	Users may set the remaining amount arbitrarily due to the absence of the final amount in the admin signature.
OS-AUR-ADV-02	Medium	Resolved	Centralization risk in the protocol.
OS-AUR-ADV-03	Low	Resolved	`_getTokenIndex` returns the incorrect index.
OS-AUR-ADV-04	Low	Resolved	Adding `nonReentrant` to `deposit` and `withdraw` may prevent reenterancy scenarios.

© 2023 Otter Audits LLC. All Rights Reserved.

OS-AUR-ADV-00 [med]| Incorrect Account And Bump

Description

`Withdraw_v2` checks the `withdraw_from_burner_ta` variable to determine whether the withdrawal will occur from `burn_ta` or `vault_ta`. However, if `withdraw_from_burner_ta` is set to `true`, it utilizes `vault_ta` and `vault_bump` instead of `burn_ta` and `burn_bump`. This may result in a runtime error and a denial of service.

Remediation

Replace `vault_ta` and `vault_bump` with `burn_ta` and `burn_bump` respectively.

programs/casier/src/lib.rs DIFF

```
@@ -242,11 +242,11 @@ pub mod casier {
                    CpiContext::new_with_signer(
                        ctx.accounts.token_program.to_account_info(),
                        anchor_spl::token::Transfer {
-                           from: ctx.accounts.vault_ta.to_account_info(),
+                           from: ctx.accounts.burn_ta.to_account_info(),
                            to: ctx.accounts.user_ta.to_account_info(),
-                           authority: ctx.accounts.vault_ta.to_account_info(),
+                           authority: ctx.accounts.burn_ta.to_account_info(),
                        },
-                       &[&[ctx.accounts.mint.key().as_ref(), &[vault_bump]]],
+                       &[&[ctx.accounts.mint.key().as_ref(), &[burn_bump]]],
                    ),
                    withdraw_amount.into(),
                )?;
```

Patch

Fixed in 91217b3 by replacing the `vault_ta` and `vault_bump` with `burn_ta` and `burn_bump` respectively.

© 2023 Otter Audits LLC. All Rights Reserved.

OS-AUR-ADV-01 [med]| Signature Omits Remaining Amount

Description

`withdraw` in `SyncSpace` verifies the validity of the admin signature utilizing the `requireAdminSignature` modifier, then transfers tokens. However, since `final_amount` is not included in the signature, users may pass their preferred `final_amount` value along with the admin signature, manipulating the remaining amount.

SnycSpace.sol SOLIDITY

```
    function withdraw(
        uint256 id,
        address tokenAddress,
        uint256 amount,
        uint256 finalAmount,
        uint256 nonce,
        SignerData memory sd
    )
        public
        whenNotPaused
        requireNonce(nonce)
        requireAdminSignature(sd, id, tokenAddress, amount, 1)
```

Remediation

Include the `final_amount` parameter as part of the data signed by the admin during signature verification to prevent manipulation of the remaining amount.

Patch

Fixed in 7126c8d by including the `final_amount` parameter in the signed data from the admin.

© 2023 Otter Audits LLC. All Rights Reserved.

OS-AUR-ADV-02 [med]| Centralization Risk

Description

There are a few sections where strict checks are not implemented since every transaction requires an admin signature and is validated off-chain.

1. `withdraw_v2` receives the `final_amount` and utilizes it to specify the remaining amount in the locker after the withdrawal. However, it does not check if `final_amount` is less than or equal to `locker.amounts - withdraw_amount`.
2. `deposit` transfers tokens from `user_ta` to `vault_ta` for locking and recording the transferred amount in the locker. The validity of `vault_ta` is assessed by verifying the owner and mint. However, this may be bypassed by creating a token account in their program that satisfies the condition. With `vault_ta` under the ownership of the attacker, they may increase the amount in the locker without locking their own tokens.
3. `SyncSpace`, `_withdrawERC20` and `_withdrawERC1155` do not include a validation check to ensure that the amount transferred is less than or equal to the user's balance. It is recommended to add a validation check during the execution of the withdrawal process to ensure that the amount withdrawn is within the limits of the user's available balance.
4. No seed checks are implemented for `vault_ta` and `burn_ta` in `WithdrawV2`. Additionally, instead of utilizing a canonical bump, they rely on user input. This may result in the scattering of funds across multiple token accounts as users exploit multiple valid bumps within a set of seeds.

Remediation

1. Verify that `final_amount <= vault.amount - withdraw_amount`.
2. Include a seed check for `vault_ta` in the `WithdrawV2` instruction. This guarantees that the token account utilized for the withdrawal is associated with the correct program, preventing manipulation by an attacker.
3. Validate if the amount is less than or equal to the balance.
4. Utilize a canonical bump instead of user input and check the seeds.

Patch

The reported issues have been accepted by mentioning corresponding checks for them and incorporated into the off-chain.

© 2023 Otter Audits LLC. All Rights Reserved.

OS-AUR-ADV-03 [low] | Inaccurate Index Searching

Description

In `ERC1155TokensByAccountUpgradeable`, the `_tokensByAccount` mapping stores the token IDs and quantities associated with each address. The storage mechanism uses a dynamic array to store token IDs and quantities by pushing them consecutively to ensure contiguous indices.

When searching for the index of a specific ID held by an account, `_getTokenIndex` iterates through all array indices to locate the desired index. However, treating the quantity as a token ID and iterating through it results in incorrect results.

Remediation

Apply the following changes:

- `_afterTokenTransfer`
  - If the token balance of `from` is zero, remove the ID from `_tokenByAccount`.
- `_beforeTokenTransfer`
  - If the token balance of `to` is zero, add the ID to `_tokenByAccount`.

As a result, `_tokenByAccount` no longer needs to store the token amount.

ERC1155Upgradeable.sol SOLIDITY

```
// Mapping from token ID to account balances
mapping(uint256 => mapping(address => uint256)) private _balances;
```

Patch

Fixed in commit 7126c8d by implementing the suggested solution. The `_tokenByAccount` has been adjusted to solely store token IDs, while the token amounts are now retrieved from `balanceOf`.

© 2023 Otter Audits LLC. All Rights Reserved.

OS-AUR-ADV-04 [low] | Prevent Reentrancy

Description

In the SyncSpace contract, `deposit` and `withdraw` do not have `ReentrancyGuard`. Therefore, if a user possesses multiple admin signatures satisfying the condition, reentrancy may occur.

Remediation

Utilize the `nonReentrant` modifier for `deposit` and `withdraw`.

SnycSpace.sol DIFF

```
@@ -196,6 +196,7 @@ contract SyncSpace is
         whenNotPaused
         requireNonce(nonce)
         requireAdminSignature(sd, id, tokenAddress, amount, 0)
+        nonReentrant
     {
         _nonce[msg.sender] += 1;
         if (tokenAddress == address(0)) {
@@ -251,6 +252,7 @@ contract SyncSpace is
         whenNotPaused
         requireNonce(nonce)
         requireAdminSignature(sd, id, tokenAddress, amount, 1)
+        nonReentrant
     {
         _nonce[msg.sender] += 1;
         if (tokenAddress == address(0)) {
```

Patch

Fixed in commit 7126c8d by utilizing the `nonReentrant` modifier for `deposit` and `withdraw`.

© 2023 Otter Audits LLC. All Rights Reserved.

05 | General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may lead to security issues in the future.

ID	Description
OS-AUR-SUG-00	`WithdrawV2` does not include an address check for `user_ta`.
OS-AUR-SUG-01	Executing `_withdrawERC20` with a zero transfer wastes gas.
OS-AUR-SUG-02	Requesting a user to input the desired number of mints increases efficiency.
OS-AUR-SUG-03	Declare string constants with a descriptive name for seeds.
OS-AUR-SUG-04	`operationType` utilizes hard-coded values instead of constants.
OS-AUR-SUG-05	Remove unutilized functions and instructions.

© 2023 Otter Audits LLC. All Rights Reserved.

OS-AUR-SUG-00 | Add Seed Check

Description

`WithdrawV2` does not perform any checks for `user_ta`. To mitigate potential attacks, implement an address check for `user_ta` using a program and seeds instead of wrapping it as `UncheckedAccount`.

Remediation

Add seeds and bumps to the macro of `user_ta`.

programs/casier/src/lib.rs RUST

```rust
pub struct WithdrawV2<'info> {
    #[account(
        mut,
        seeds = [b"user_ta".as_ref(), token_program.key().as.ref()],
        bump,
    )]
    pub user_ta: AccountInfo<'info>,
```

© 2023 Otter Audits LLC. All Rights Reserved.

OS-AUR-SUG-01 | Avoid Zero Transfer

Description

`_withdrawERC20` of `SyncSpace` verifies whether or not `ssErc20Balance` is less than the amount being transferred. If the condition is met, it selects `ssErc20Balance` as the transfer amount.

Verifying that `ssErc20Balance` equals zero before executing `token.transfer(msg.sender, ssErc20Balance)` would prevent zero transfers from occurring, which saves gas.

Remediation

Verify if `ssErc20Balance` is greater than zero before executing the transfer.

SnycSpace.sol DIFF

```
@@ -279,7 +279,7 @@ contract SyncSpace is
         if (ssErc20Balance < amount) {
             require(isAury, "Insufficient ERC20 balance");
             token.mint(msg.sender, amount - ssErc20Balance);
-            token.transfer(msg.sender, ssErc20Balance);
+            if (ssErc20Balance != 0) token.transfer(msg.sender, ssErc20Balance);
         } else {
             token.transfer(msg.sender, amount);
         }
```

© 2023 Otter Audits LLC. All Rights Reserved.

OS-AUR-SUG-02 | Account Size Calculation

Description

In `init_locker`, the allocated space for an account is determined by the passed `space`. Receiving the number of potential mints in `Locker` as user input and then calculating the space based on that would increase efficiency. Storing the number of mints in `Locker` instead of the allocated space would allow `deposit` to reference that and raise an error when reaching the space limit.

Remediation

Implement the following example since the number of mints affects the vector sizes of `mints` and `amount`. Moreover, storing the number of mints in `Locker` may be referenced when checking against the maximum limit.

programs/casier/src/state.rs RUST

```rust
pub const ANCHOR_SIZE_DISCRIMINATOR : u64 = 8;
pub const ANCHOR_SIZE_U8: u64 = 1;
pub const ANCHOR_SIZE_U64: u64 = 8;
pub const ANCHOR_SIZE_PUBKEY: u64 = 32;
pub const ANCHOR_SIZE_VEC_INTERNAL: u64 = 4;

#[derive(Accounts)]
#[instruction(_mint: u64)]
pub struct InitLocker<'info> {
    #[account(
        init,
        seeds = [ owner.key().as_ref() ],
        bump,
        payer = owner,
        space = (ANCHOR_SIZE_DISCRIMINATOR + ANCHOR_SIZE_PUBKEY +
    ↪  (ANCHOR_SIZE_VEC_INTERNAL + ANCHOR_SIZE_PUBKEY * _mint) +
    ↪  (ANCHOR_SIZE_VEC_INTERNAL + ANCHOR_SIZE_U64 * _mint) + ANCHOR_SIZE_U8 +
    ↪  ANCHOR_SIZE_U64) as usize
    )]
    pub locker: Account<'info, Locker>,
    #[account(mut)]
    pub owner: Signer<'info>,
    pub system_program: Program<'info, System>,
    pub rent: Sysvar<'info, Rent>,
}
```

© 2023 Otter Audits LLC. All Rights Reserved.

OS-AUR-SUG-03 | Use Prefix String Seed

Description

Utilizing prefix string constants for each type of address derivation, such as `locker`, `user_ta`, `burn_ta`, and `vault_ta`, would be a proactive approach for enhancing code clarity and safety. For example, declare constants named `VAULT_PREFIX`, `BURN_PREFIX`, and `LOCKER_PREFIX` and add them to the seeds.

Remediation

Modify the seeds as follows to include prefix string:

programs/casier/src/state.rs RUST

```rust
#[constant]
pub const LOCKER_PREFIX: &'static [u8] = b"LOCKER_PREFIX";

#[derive(Accounts)]
pub struct WithdrawV2<'info> {
    #[account(
        seeds = [ b"config".as_ref() ],
        bump,
        has_one = admin,
        constraint = !config.is_frozen
    )]
    pub config: Account<'info, Config>,
    #[account(
        mut,
        seeds = [LOCKER_PREFIX],
        bump,
    )]
    pub locker: Account<'info, Locker>,
```

© 2023 Otter Audits LLC. All Rights Reserved.

OS-AUR-SUG-04 | Replace Hard-Coded Values With Constants

Description

In `SyncSpace`, the `operationType` variable passed to the `requireAdminSignature` modifier is currently hard-coded as zero for deposits and one for withdrawals. Utilizing constant variables instead of hard-coding zero and one as `operationType` values would enhance the readability and maintainability of the code.

Remediation

Replace hard-coded values with constants with meaningful names.

SnycSpace.sol DIFF

```
@@ -20,6 +20,8 @@ contract SyncSpace is
     bytes32 public constant PAUSER_ROLE = keccak256("PAUSER_ROLE");
     bytes32 public constant UPGRADER_ROLE = keccak256("UPGRADER_ROLE");
     bytes32 public constant TRANSFER_ADMIN = keccak256("TRANSFER_ADMIN");
+    uint8 public constant OPERATION_TYPE_DEPOSIT = 0;
+    uint8 public constant OPERATION_TYPE_WITHDRAW = 1;

     // account => tokenAddress => balance
     mapping(address => mapping(address => uint256)) private _erc20Balance;
@@ -195,7 +197,7 @@ contract SyncSpace is
         public
         whenNotPaused
         requireNonce(nonce)
-        requireAdminSignature(sd, id, tokenAddress, amount, 0)
+        requireAdminSignature(sd, id, tokenAddress, amount,
    ↪  OPERATION_TYPE_DEPOSIT)
     {
         _nonce[msg.sender] += 1;
         if (tokenAddress == address(0)) {
@@ -250,7 +252,7 @@ contract SyncSpace is
         public
         whenNotPaused
         requireNonce(nonce)
-        requireAdminSignature(sd, id, tokenAddress, amount, 1)
+        requireAdminSignature(sd, id, tokenAddress, amount,
    ↪  OPERATION_TYPE_WITHDRAW)
     {
         _nonce[msg.sender] += 1;
         if (tokenAddress == address(0)) {
```

© 2023 Otter Audits LLC. All Rights Reserved.

OS-AUR-SUG-05 | Eliminate Unutilized Code

Description

Eliminate unutilized code for maintenance and clarity.

Remediation

Remove the sections of code listed below:

- `lib.rs`
  - `initialize`
  - `increase_locker_size`
- `state.rs`
  - `Initialize`
  - `IncreaseLockerSize`
  - `Withdraw`
  - `WithdrawAndBurn`
- `utils.rs`
  - `spl_init_token_account2`

© 2023 Otter Audits LLC. All Rights Reserved.

A | Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings can be found in the General Findings section.

Critical	Vulnerabilities that immediately lead to loss of user funds with minimal preconditions Examples: • Misconfigured authority or access control validation • Improperly designed economic incentives leading to loss of funds
High	Vulnerabilities that could lead to loss of user funds but are potentially difficult to exploit. Examples: • Loss of funds requiring specific victim interactions • Exploitation involving high capital requirement with respect to payout
Medium	Vulnerabilities that could lead to denial of service scenarios or degraded usability. Examples: • Malicious input that causes computational limit exhaustion • Forced exceptions in normal user flow
Low	Low probability vulnerabilities which could still be exploitable but require extenuating circumstances or undue risk. Examples: • Oracle manipulation with large capital requirements and multiple transactions
Informational	Best practices to mitigate future security risks. These are classified as general findings. Examples: • Explicit assertion of critical internal invariants • Improved input validation

© 2023 Otter Audits LLC. All Rights Reserved.

B | Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the implementation of the program requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of sum, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to get a comprehensive understanding of the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that the other missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2023 Otter Audits LLC. All Rights Reserved.